Model Leakage: Attacking and Defending a Fine-tuned GPT-2

PLACEHOLDER / DRAFT (2026–2027 edition). Scope and resources to be finalised. This brief sketches the intended project; experiments, dataset, and grading rubric will be completed before release.

1 Introduction

Fine-tuning a language model on a private corpus does not only teach it a task — it can memorise the training data itself. When that corpus contains personal information, the fine-tuned model becomes a privacy liability: an adversary with query (or weight) access may recover training records. This project studies that threat end to end on a controllable, small-scale setup: a **GPT-2 small** model fine-tuned on a corpus seeded with synthetic "personal" records, where you play both *attacker* and *defender*.

This project is the deep-dive companion to Assignment 4 (generative fine-tuning, canaries, and prefix-extraction), extending it from a scripted exercise into an open attack/defence study.

Goal Characterise *when* and *how much* a fine-tuned GPT-2 leaks its training data, mount concrete attacks to quantify the leak, and evaluate defences that reduce it — measuring the privacy/utility trade-off throughout.

2 Research Questions

RQ1 (Memorisation): How does training-data memorisation in fine-tuned GPT-2 scale with record repetition count, model size, and number of fine-tuning epochs?

RQ2 (Attack): How effectively can membership-inference and extraction attacks recover personal records, and what signal (loss, perplexity ratio, prefix-prompting) drives them?

RQ3 (Defence): Which defences — deduplication, early stopping, output filtering, or differentially private fine-tuning (DP-SGD) — reduce leakage, and at what cost to task utility?

3 Experimental Setup (to finalise)

1. **Data & canaries.** Fine-tune GPT-2 small on a provided raw corpus into which synthetic personal records ("canaries", e.g. names + fake IDs/phone numbers) are inserted at known, varying repetition frequencies.
2. **Memorisation curve.** Measure held-out perplexity before/after fine-tuning and plot extraction/exposure as a function of canary repetition count (link the result back to A4).
3. **Attacks.** Implement (a) a loss/perplexity-threshold *membership-inference* attack and (b) a *prefix-prompting extraction* attack; report attack success per canary frequency and a leakage metric (e.g. exposure or extraction rate).
4. **Defences.** Apply at least two of: training-set deduplication, early stopping, decoding-time filtering, and DP-SGD fine-tuning (e.g. via Opacus). Re-run the attacks and report the privacy/utility trade-off.
5. **Analysis.** Discuss which records leak most and why, and what the repetition threshold implies for training on real private data.

Resources GPT-2 small (HuggingFace `transformers`); runs on Colab / a 16 GB GPU. DP-SGD via Opacus. Compute is modest by design.

Links to the course Assignment 4 (canaries + extraction), lecture *Generative Modelling / Alignment*, explainer videos #04 Word Embeddings, #09 From GPT to ChatGPT, #12 How an LLM is Trained.